

AI Exam Master Guide — Chapters 1 & 2.1 (+2.6 Fuzzy Logic)

Everything from the illustrated deep-dive **plus** fully worked answers to real past-paper questions found from other colleges' exams — organized topic-by-topic so you can study this one document start to finish. Diagrams included wherever they genuinely help.

Contents

- 1.1 Intelligent Agents
- 1.2 Advanced Search Techniques (+ past-paper: A* limitations & IDA*)
- 1.3 Local Search (+ past-paper: Simulated Annealing numerical)
- 1.4 Games
- 2.1 First-Order Logic (+ past-paper: CNF conversions, contrapositive truth table, full resolution proof)
- 2.6 Fuzzy Logic (+ past-paper: Mamdani FIS, traffic light system)
- Self-check checklist

1.1 Intelligent Agents

An agent is anything that **looks at the world (sensors)** and **does something about it (actuators)**. Think of it as a loop: see → think → act → repeat. There are 5 types, each a smarter upgrade of the last.



Type	What it does	Weakness
Simple reflex	If condition X, do action Y. No memory.	Gets lost without full visibility
Model-based reflex	Remembers past state to fill in what it can't see	Still doesn't plan ahead
Goal-based	Chooses actions to reach a goal	Doesn't compare how good different paths are

Utility-based	Scores outcomes, picks the highest	Needs a well-designed utility function
Learning	Improves through experience over time	Needs data/feedback to learn from

EXAM STRUCTURE

If asked "Explain types of intelligent agents": (1) one line defining agent = perceive + act, (2) mention PEAS if relevant, (3) go through the 5 types with a tiny example each, (4) end comparing simplest vs smartest.

REAL PAST-PAPER QUESTION — FULL EXAM ANSWER

Short note: Learning Agent — appeared as a "write short notes" option in a real past paper.

A learning agent improves its performance over time through experience, rather than relying on fixed rules. It has 4 components:

- **Performance element** — selects external actions (this is essentially the "agent" from the other 4 types, doing the actual acting)
- **Learning element** — responsible for making improvements, using feedback to modify the performance element
- **Critic** — gives feedback on how the agent is doing, judged against a fixed performance standard (since the environment itself doesn't hand out grades)
- **Problem generator** — suggests exploratory actions that might not be optimal short-term but lead to better learning (e.g., trying a new route instead of the known best one, to discover something even better)

Example: a spam filter starts with basic rules, but the learning element updates its filtering model based on emails the user marks as spam/not-spam — the critic uses that user feedback as its performance standard.

1.2 Advanced Search Techniques

Normal search (BFS/DFS) explores blindly. Advanced techniques use extra info (heuristics) or smarter directions to search faster.

- **A* Search:** picks next node using $f(n) = g(n) + h(n)$. $g(n)$ = cost so far, $h(n)$ = estimated cost to goal. Optimal if $h(n)$ never overestimates (admissible).
- **Bidirectional Search:** searches from start AND goal simultaneously, meets in the middle — roughly halves search depth.
- **IDA*:** like A* but memory-light — repeated depth-first searches bounded by increasing $f(n)$ cost limits instead of depth.
- **Online Search:** agent doesn't know the map in advance — moves first, observes, then decides next step.

EXAM STRUCTURE

For "Explain A* with example": define $f(n)=g(n)+h(n)$, state the admissibility condition, then do a small worked trace — examiners reward showing the calculation, not just the definition.

WORKED EXAMPLE

A* trace: Edges A-B(1), A-C(4), B-D(5), C-D(1). Heuristics: $h(A)=5$, $h(B)=4$, $h(C)=1$, $h(D)=0$.

$f(A)=0+5=5 \rightarrow$ expand A $\rightarrow$ B: $f=1+4=5$, C: $f=4+1=5$ (tie)

Expand B $\rightarrow$ D via B: $f=1+5+0=6$

Expand C $\rightarrow$ D via C: $f=4+1+0=5$

Since $5 < 6$, A* picks path **A→C→D**, total cost 5.

REAL PAST-PAPER QUESTION — FULL EXAM ANSWER

Q: What are the limitations of A* search? How does Iterative Deepening A* (IDA*) resolve these problems? [7 marks]

Limitations of A*:

- A* keeps every generated node in memory (both the frontier and the explored set) so it can always expand the most promising one. In large search spaces, this list grows exponentially with search depth.
- Memory usage becomes the practical bottleneck long before time does — A* often runs out of memory on large problems even though it would eventually find the answer given enough RAM.

How IDA* resolves this:

- IDA* combines the idea of *iterative deepening* (repeated depth-first searches with an increasing limit) with A*'s cost function, but bounds by **$f(n)$ cost** instead of depth.
- Each iteration performs a depth-first search, pruning (cutting off) any branch where $f(n)$ exceeds the current threshold.
- If no solution is found, the threshold is raised to the smallest $f(n)$ value that was cut off, and the search restarts from scratch.
- Because it's depth-first at heart, IDA* only needs to store the current path — memory usage is linear in the depth of the search, not exponential.

Trade-off to mention: IDA* re-explores nodes across iterations (extra time cost) in exchange for far better memory efficiency — a classic time-vs-memory trade-off worth stating for full marks.

1.3 Local Search

Local search only cares about the final answer, not the path — used for optimization problems.

- **Simulated Annealing:** like hill climbing, but sometimes accepts a worse move on purpose (more often early on, controlled by a cooling "temperature") to escape local optima.
- **Local Beam Search:** tracks k states at once; all generate neighbors, best k of the whole pool survive to the next round.
- **Partially observable environments:** agent tracks a **belief state** — the set of states it could currently be in — since it can't fully see the world.

EXAM STRUCTURE

Compare/contrast questions want a table. Use: states tracked, how it escapes local optima, role of randomness, main risk.

REAL PAST-PAPER QUESTION — FULL EXAM ANSWER

Q: How does Simulated Annealing resolve the problems of local hill-climbing search? Apply Simulated Annealing to optimize $f(x) = -x^2 + 4x$. [8 marks]

How SA fixes hill climbing: Hill climbing always moves to a strictly better neighbor and stops the moment no better neighbor exists — this traps it at the first peak found, even if it's only a **local** maximum. Simulated Annealing fixes this by occasionally accepting a **worse** move, with acceptance probability $P = e^{-(\Delta f / T)}$, where Δf is how much worse the move is and T is a temperature that starts high and gradually cools. Early on (high T), the search explores freely, even taking downhill moves — this is what lets it escape local optima. As T cools toward 0, it behaves more and more like strict hill climbing, converging on a good (ideally global) solution.

Algorithm steps:

1. Start at an initial x , with a high temperature T
2. Generate a random neighbor x' (e.g., $x \pm 1$)
3. Compute $\Delta f = f(x') - f(x)$
4. If $\Delta f > 0$ (better) → always accept
5. If $\Delta f \leq 0$ (worse) → accept anyway with probability $e^{-(\Delta f / T)}$
6. Decrease T (e.g., $T \leftarrow 0.9T$) and repeat until T is close to 0

Applying it to $f(x) = -x^2 + 4x$ (start $x=0$, $T=10$, cooling factor 0.9):

Step	x	$f(x)$	x'	$f(x')$	Δf	T	Decision
1	0	0	1	3	+3	10	Accept (better) → $x=1$
2	1	3	2	4	+1	9	Accept (better) → $x=2$
3	2	4	3	3	-1	8.1	Accept with $P=e^{-(1/8.1)} \approx 0.88$ → $x=3$
4	3	3	2	4	+1	7.3	Accept (better) → $x=2$
5	2	4	1 or 3	3	-1	6.6	Accept prob. shrinks as T cools — search settles at $x=2$

By calculus: $f'(x) = -2x+4 = 0 \rightarrow x = 2$ gives the maximum, **$f(2) = 4$** . The trace above converges there as T cools, while showing SA briefly explored a worse state ($x=3$) early on — something plain hill climbing could never do.

Note: if your copy of the question shows " $f(x) = -x^2 + 4xz$ " with a second variable z , that's very likely a transcription slip — the standard version of this exact textbook question uses a single variable, as solved above. Same method applies either way.

WORKED EXAMPLE

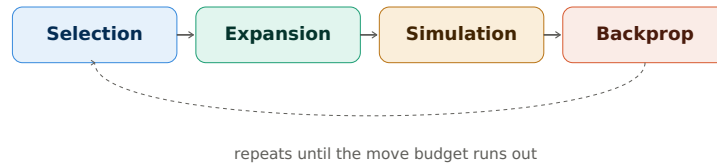
OR-option: Local Beam Search idea

Instead of tracking one current state, Local Beam Search tracks **k states in parallel**. At each step, all k states

generate their neighbors, and the best k states out of the *entire combined pool* are kept for the next round — the k states share information, unlike running k independent hill-climbing searches. This lets the search concentrate effort on the most promising regions found so far, though it risks losing diversity if all k states converge to the same area (a form of local optimum for the whole beam).

1.4 Games

- **Minimax:** MAX wants highest score, MIN wants lowest; both assumed to play optimally, solved backward from the end.
- **Alpha-Beta Pruning:** skips branches that can't possibly change the outcome — same result as minimax, less computation.
- **MCTS:** used when the tree is too huge for minimax. Loops 4 steps (see diagram).
- **Expectiminimax:** handles randomness (dice/cards) by adding chance nodes that average outcomes by probability.



WORKED EXAMPLE

Alpha-beta rule of thumb: prune a branch once its best possible value can no longer change the decision at a parent above it, because the parent already has a guaranteed better option elsewhere.

2.1 First-Order Logic (FOL) and Reasoning ☆

Why is CNF required?

Resolution — the standard proof technique in FOL — only works on sentences in **Conjunctive Normal Form** (a conjunction of clauses, each a disjunction of literals). Converting every knowledge-base sentence to CNF gives inference algorithms one uniform, mechanical format, so resolution can be applied consistently regardless of how a sentence was originally phrased.

Steps to convert to CNF (memorize this order)

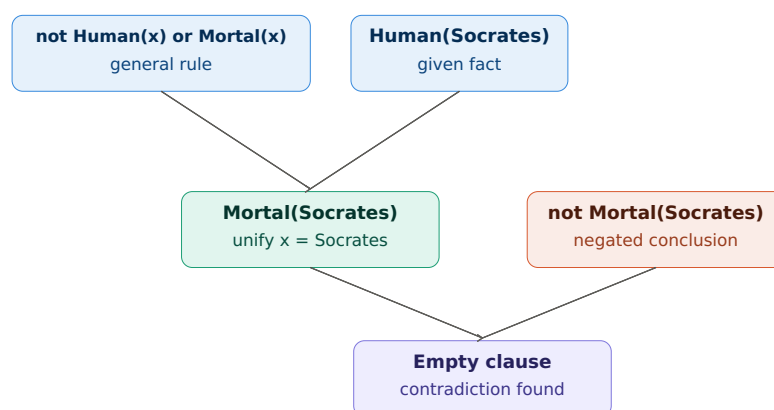
1. Eliminate biconditional $\Leftrightarrow$: $A \Leftrightarrow B \rightarrow (A \Rightarrow B) \wedge (B \Rightarrow A)$
2. Eliminate implication $\Rightarrow$: $A \Rightarrow B \rightarrow \neg A \vee B$
3. Move negation inward (De Morgan's, double negation, quantifier flip)
4. Standardize variables apart
5. Move quantifiers to the front (prenex form)
6. Skolemize — remove $\exists$ by replacing with a Skolem constant/function
7. Drop all $\forall$ (implied from here)
8. Distribute $\vee$ over $\wedge$ for final clause form
9. Split conjunctions into separate clauses

Unification

`Likes(x, Mary)` and `Likes(John, y)` unify with substitution `{x/John, y/Mary}` — the Most General Unifier (MGU).

Resolution Refutation — the proof method

1. Convert all premises to CNF
2. **Negate the conclusion**, convert it too, add to the clause set
3. Find two clauses with complementary literals that unify
4. Resolve — combine, cancel the complementary pair
5. Repeat until you derive the **empty clause** $\rightarrow$ contradiction $\rightarrow$ original statement is TRUE



WORKED EXAMPLE

Simplest case — Socrates: Given $\forall x(\text{Human}(x) \Rightarrow \text{Mortal}(x))$ and $\text{Human}(\text{Socrates})$, prove $\text{Mortal}(\text{Socrates})$.
Clause 1: $\neg \text{Human}(x) \vee \text{Mortal}(x)$. Clause 2: $\text{Human}(\text{Socrates})$. Negated conclusion Clause 3: $\neg \text{Mortal}(\text{Socrates})$.
Resolve 1+2 ($x=\text{Socrates}$) $\rightarrow \text{Mortal}(\text{Socrates})$. Resolve that with Clause 3 $\rightarrow$ empty clause. Contradiction proves $\text{Mortal}(\text{Socrates})$ TRUE. ■

REAL PAST-PAPER QUESTION — FULL EXAM ANSWER

Q: Transform the following into CNF. [part of a 7+8 mark question]

(i) "Everyone who loves all animals is loved by someone."

FOL: $\forall x [\forall y (\text{Animal}(y) \Rightarrow \text{Loves}(x,y)) \Rightarrow \exists y \text{ Loves}(y,x)]$

Step 1 — eliminate inner $\Rightarrow$: $\forall x [\forall y (\neg \text{Animal}(y) \vee \text{Loves}(x,y)) \Rightarrow \exists y \text{ Loves}(y,x)]$

Step 2 — eliminate outer $\Rightarrow$: $\forall x [\neg \forall y (\neg \text{Animal}(y) \vee \text{Loves}(x,y)) \vee \exists y \text{ Loves}(y,x)]$

Step 3 — push $\neg$ inward ($\neg \forall y \rightarrow \exists y \neg$): $\forall x [\exists y (\text{Animal}(y) \wedge \neg \text{Loves}(x,y)) \vee \exists y \text{ Loves}(y,x)]$

Step 4 — standardize variables apart (rename 2nd $\exists y$ to $\exists z$): $\forall x [\exists y (\text{Animal}(y) \wedge \neg \text{Loves}(x,y)) \vee \exists z \text{ Loves}(z,x)]$

Step 5 — quantifiers to front: $\forall x \exists y \exists z [(\text{Animal}(y) \wedge \neg \text{Loves}(x,y)) \vee \text{Loves}(z,x)]$

Step 6 — Skolemize (y,z depend on $x \rightarrow$ Skolem functions $F(x)$, $G(x)$): $(\text{Animal}(F(x)) \wedge \neg \text{Loves}(x,F(x))) \vee \text{Loves}(G(x),x)$

Step 7 — drop $\forall x$

Step 8 — distribute $\vee$ over $\wedge$: $(\text{Animal}(F(x)) \vee \text{Loves}(G(x),x)) \wedge (\neg \text{Loves}(x,F(x)) \vee \text{Loves}(G(x),x))$

Final CNF:

Clause 1: $\text{Animal}(F(x)) \vee \text{Loves}(G(x), x)$

Clause 2: $\neg \text{Loves}(x, F(x)) \vee \text{Loves}(G(x), x)$

(ii) $P \rightarrow ((Q \wedge \neg R) \leftrightarrow S)$

Step 1 — eliminate $\leftrightarrow$ ($A=B \wedge B=A$): $(A \Rightarrow B) \wedge (B \Rightarrow A) = ((Q \wedge \neg R) \Rightarrow S) \wedge (S \Rightarrow (Q \wedge \neg R))$

Step 2 — eliminate all $\Rightarrow$: $\neg P \vee [(\neg(Q \wedge \neg R) \vee S) \wedge (\neg S \vee (Q \wedge \neg R))]$

Step 3 — push $\neg$ inward ($\neg(Q \wedge \neg R) \rightarrow \neg Q \vee R$): $\neg P \vee [(\neg Q \vee R \vee S) \wedge (\neg S \vee (Q \wedge \neg R))]$

Step 4 — distribute $\vee$ over $\wedge$ (outer, then inner):

$\neg P \vee \neg Q \vee R \vee S$

$\neg P \vee \neg S \vee (Q \wedge \neg R) \rightarrow$ splits into: $(\neg P \vee \neg S \vee Q) \wedge (\neg P \vee \neg S \vee \neg R)$

Final CNF:

Clause 1: $\neg P \vee \neg Q \vee R \vee S$

Clause 2: $\neg P \vee \neg S \vee Q$

Clause 3: $\neg P \vee \neg S \vee \neg R$

REAL PAST-PAPER QUESTION — FULL EXAM ANSWER

Q: Ram attempted two interviews. Company A said "If you are qualified, you will get the job." Company B said "If you did not get the job, you were not qualified." Do both statements say the same thing? Answer using a truth-table approach. [7 marks]

Let Q = you are qualified, J = you get the job.

Company A: $Q \Rightarrow J$. Company B: $\neg J \Rightarrow \neg Q$ (this is the *contrapositive* of A).

Q	J	$Q \Rightarrow J$	$\neg J \Rightarrow \neg Q$
T	T	T	T
T	F	F	F
F	T	T	T
F	F	T	T

Conclusion: All four rows match exactly — the two statements are logically equivalent. This holds in general because an implication is always equivalent to its contrapositive: $A \Rightarrow B \equiv \neg B \Rightarrow \neg A$. Company A and B are expressing the identical rule, just phrased in opposite directions.

REAL PAST-PAPER QUESTION — FULL EXAM ANSWER

Q: Prove using resolution — "Anyone who is lucky wins the lottery and passes all exams, and anyone who passes exams and wins the lottery is happy. Given John is lucky (but didn't study), prove John is happy." [full worked proof]

Step 1 — Predicates & FOL

Study(x), Lucky(x), PassExams(x), WinsLottery(x), Happy(x)

- 1. $\forall x (PassExams(x) \wedge WinsLottery(x) \Rightarrow Happy(x))$
- 2. $\forall x (Study(x) \vee Lucky(x) \Rightarrow PassExams(x))$
- 3. $\neg Study(John) \wedge Lucky(John)$
- 4. $\forall x (Lucky(x) \Rightarrow WinsLottery(x))$

To prove: Happy(John)

Step 2 — CNF conversion (full working)

Sentence 1: eliminate $\Rightarrow \rightarrow \neg(Pass(x) \wedge Wins(x)) \vee Happy(x) \rightarrow$ De Morgan's $\rightarrow$

Clause A: $\neg PassExams(x) \vee \neg WinsLottery(x) \vee Happy(x)$

Sentence 2: eliminate $\Rightarrow \rightarrow \neg(Study(x) \vee Lucky(x)) \vee Pass(x) \rightarrow$ De Morgan's $\rightarrow (\neg Study(x) \wedge \neg Lucky(x)) \vee Pass(x) \rightarrow$ distribute $\rightarrow$

Clause B1: $\neg Study(x) \vee PassExams(x)$ Clause B2: $\neg Lucky(x) \vee PassExams(x)$

Sentence 3: split conjunction $\rightarrow$

Clause C1: $\neg Study(John)$ Clause C2: $Lucky(John)$

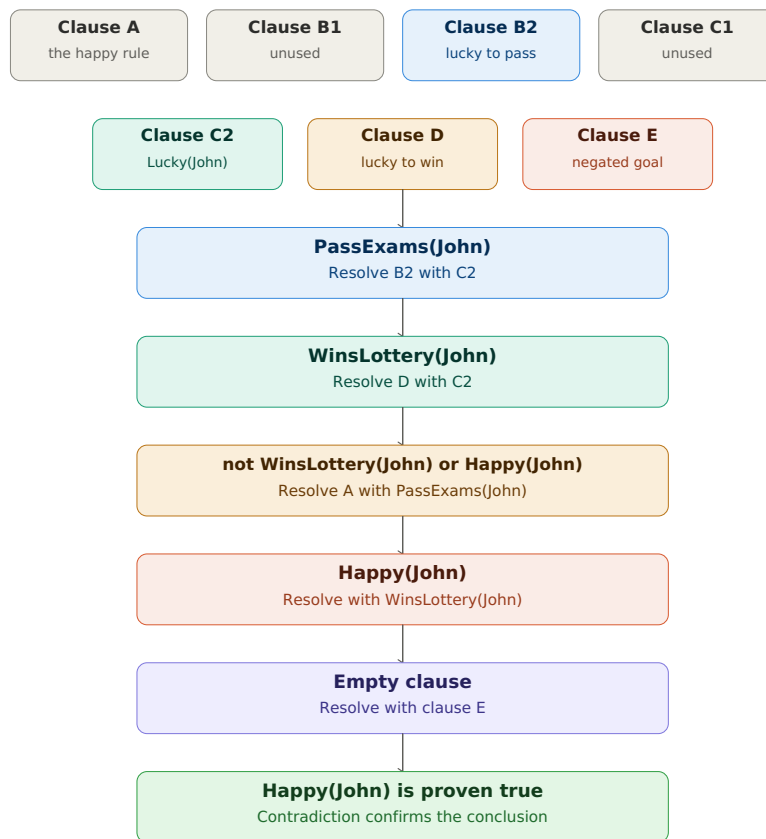
Sentence 4: eliminate $\Rightarrow \rightarrow$

Clause D: $\neg Lucky(x) \vee WinsLottery(x)$

Negate conclusion: Clause E: $\neg Happy(John)$

Step 3 — Resolution, step by step

#	Resolve	Unify	Result
1	B2 ($\neg Lucky(x) \vee Pass(x)$) with C2 ($Lucky(John)$)	$x=John$	F: PassExams(John)
2	D ($\neg Lucky(x) \vee Wins(x)$) with C2 ($Lucky(John)$)	$x=John$	G: WinsLottery(John)
3	A ($\neg Pass(x) \vee \neg Wins(x) \vee Happy(x)$) with F	$x=John$	H: $\neg WinsLottery(John) \vee Happy(John)$
4	H with G ($WinsLottery(John)$)	—	I: Happy(John)
5	I with E ($\neg Happy(John)$)	—	[] — empty clause



Note: Clauses B1 and C1 (the "John did not study" facts) were never needed — that's completely normal. Not every given fact has to be used in every proof; only use what resolution actually needs. On the exam paper, write out every clause (A through E) explicitly like above — that's where the marks come from, not just the final chain.

2.6 Fuzzy Logic (new section — needed for the fuzzy-system questions below)

Fuzzy logic handles **partial truth** (values between 0 and 1) instead of strict binary true/false.

Mamdani Fuzzy Inference System — 4 steps

1. **Fuzzification** — convert crisp input values into fuzzy membership degrees, using membership functions (usually triangular/trapezoidal)
2. **Rule Evaluation (Inference)** — apply fuzzy IF-THEN rules, combining conditions with fuzzy AND (min) / OR (max) to get each rule's "firing strength"
3. **Aggregation** — combine all fired rules' output fuzzy sets into one overall fuzzy output set (typically using max)
4. **Defuzzification** — convert that fuzzy output into a single crisp number (commonly via the **centroid method**)

REAL PAST-PAPER QUESTION — FULL EXAM ANSWER

Q: Illustrate the Mamdani Fuzzy Inference System with a suitable example. [8 marks]

Use the 4-step structure above, then apply it to a concrete example — see the traffic-light system below, which works perfectly as "a suitable example" for this exact question.

REAL PAST-PAPER QUESTION — FULL EXAM ANSWER

Q: A city uses a fuzzy logic-based traffic control system to adjust green light duration depending on traffic density. (i) What fuzzy system could be designed? (ii) Explain how defuzzification helps in final decision-making. [7+8]

(i) Fuzzy system design:

Input variable: Traffic Density (vehicles/minute) — fuzzy sets: Low, Medium, High

Output variable: Green Light Duration (seconds) — fuzzy sets: Short, Medium, Long

Rule base:

R1: IF Density is Low THEN Duration is Short

R2: IF Density is Medium THEN Duration is Medium

R3: IF Density is High THEN Duration is Long

Worked trace: suppose crisp input = 40 vehicles/min, and fuzzification gives $\mu(\text{Low})=0.2$, $\mu(\text{Medium})=0.7$, $\mu(\text{High})=0.1$.

Rule firing strengths (direct pass-through, single-condition rules): R1→0.2, R2→0.7, R3→0.1

Aggregation: clip Short at height 0.2, Medium at 0.7, Long at 0.1, then take the union (max) of all three clipped shapes → one combined fuzzy output region.

(ii) Role of defuzzification:

Fuzzy rules only produce a *fuzzy* answer — a shape, not a number — but a real traffic controller needs one exact value to actually run the signal. Defuzzification (typically the **centroid method** — computing the center of gravity of the combined output shape) converts that fuzzy region into a single crisp number, e.g. **≈35 seconds** of green light. This is the step that makes fuzzy reasoning practically usable: it bridges "the light should probably be medium-to-long" into a concrete, actionable decision like "38 seconds."

Quick Self-Check Before the Exam

- ☐ List the 5 agent types in order and say why each improves on the last, plus the 4 components of a learning agent
- ☐ Write $f(n) = g(n) + h(n)$, explain admissibility, and state A*'s main limitation + how IDA* fixes it
- ☐ Explain why Simulated Annealing avoids getting stuck, and be able to trace it on a simple function
- ☐ Draw the 4-step MCTS loop from memory
- ☐ Do a full CNF conversion (implication, biconditional, and quantified sentences) and a full resolution proof from scratch
- ☐ Prove an implication and its contrapositive are equivalent using a truth table
- ☐ Draw and explain the 4 steps of Mamdani FIS with a worked example, including why defuzzification matters